Rival Payments — Transaction Ingestion API

This document describes the single HTTP API used to push transactions into the Rival Payments system:

- **`POST /ingest/whish/transactions`** — submit a transaction for an operating entity.

Each operating entity is identified by a **slug** (a short lowercase identifier such as `acme-retail`). Rival Payments provides you the slug for every entity you may submit transactions for — you never need to look them up yourself.

Contents

1. Connection basics — base URL, authentication, response envelope
2. Submit a transaction — request, response, idempotency
3. Error handling — status codes and retry policy
4. Example (cURL)
5. Integration summary

---

1. Connection basics

Item	Value
Base URL	`https://portal.rivalpayments.com/v1`
Protocol	HTTPS only
Content type	`application/json` (request and response)
Authentication	`Authorization: Bearer <API_KEY>` header on every request
Character encoding	UTF-8

Authentication

Every request must carry your API key in the `Authorization` header. Your key is:

```
Authorization: Bearer tsk_XXXXXXXX_REPLACE_WITH_YOUR_ISSUED_KEY
```

- This key identifies you as the `whish` source. All requests use it unchanged.
- The key is secret — treat it like a password, send it only over HTTPS, and never embed it in client-side code or logs.
- A missing, malformed, invalid, or revoked key returns **401 Unauthorized**.

Response envelope

Every JSON response uses the **same envelope**, so you can branch on one field and read the status code straight from the body:

- **Success:** `{ "success": true, "statusCode": <code>, "data": <payload> }` — your result is always under `data`.
- **Error:** `{ "success": false, "statusCode": <code>, "error": { "code", "message", "details" } }`.

`statusCode` mirrors the HTTP status line. All the examples below show the full envelope.

---

2. Submit a transaction

Request

```
POST https://portal.rivalpayments.com/v1/ingest/whish/transactions
Authorization: Bearer <API_KEY>
Content-Type: application/json
```

- Use this exact URL — `whish` is your fixed source segment. Posting to any other source path is rejected with **403 Forbidden**, since your API key is bound to `whish`.

Request body

```
{
  "companySlug": "acme-retail",
  "externalTransactionId": "WISH-2026-000123456",
  "amount": "150.00",
  "currency": "USD",
  "payload": {
    "customer": "John Doe",
    "tradingAccount": "0096170123456",
    "phone": "0096170123456",
    "note": "Top-up via Whish"
  }
}
```

Body field reference

Field	Type	Required	Rules
`companySlug`	string	**Yes**	The slug of the operating entity, as provided to you by Rival Payments.
`externalTransactionId`	string	**Yes**	Your own unique reference for this transaction. 1–200 characters. Used for de-duplication (see Idempotency).
`amount`	string (decimal)	**Yes**	The amount as a **decimal string** (never a number/float), e.g. `"150.00"`. Must be positive.
`currency`	string	**Yes**	Currency code. Currently only `USD` is supported.
`payload`	object	No (defaults to `{}`)	Descriptive details about the transaction. See below.
`idempotencyKey`	string	No	**Rarely needed.** Optional explicit idempotency key (8–120 characters). You can ignore it: `externalTransactionId` already de-duplicates your transactions (see Idempotency).

`payload` **object** — all sub-fields are optional. The following keys are recognised; **any additional keys you send are also stored verbatim** for audit:

`payload` key	Type	Max length	Meaning
`customer`	string	200	The customer who paid.
`tradingAccount`	string	200	The trading account they paid (account/number/etc.).
`phone`	string	40	A contact phone number.
`note`	string	1000	Free-text note.
(any extra keys)	any	—	Preserved as-is for audit.

Response — `201 Created`


```
{
  "success": true,
  "statusCode": 201,
  "data": {
    "id": "a7c3e9f1-2b44-4d8e-9c10-5f6a7b8c9d0e",
    "companySlug": "acme-retail",
    "sourceCode": "whish",
    "externalTransactionId": "WISH-2026-000123456",
    "amount": "150",
    "currency": "USD",
    "status": "COMPLETED",
    "customer": "John Doe",
    "tradingAccount": "0096170123456",
    "phone": "0096170123456",
    "note": "Top-up via Whish",
    "processedAt": "2026-06-04T12:34:56.000Z",
    "createdAt": "2026-06-04T12:34:56.000Z",
    "replayed": false
  }
}
```


The transaction object is under **`data`** (fields below are relative to `data`).

Response field reference

Field	Type	Description
`id`	string (UUID)	Rival Payments' reference for this transaction.
`companySlug`	string	The operating entity the transaction was submitted for.
`sourceCode`	string	Always `whish` (your source).
`externalTransactionId`	string	Echo of the reference you sent.
`amount`	string (decimal)	The recorded amount, in canonical form — trailing zeros are trimmed (`"150.00"` comes back as `"150"`). Compare numerically, not by string equality.
`currency`	string	Currency code.
`status`	string	`COMPLETED` for an accepted transaction.
`customer` / `tradingAccount` / `phone` / `note`	string | null	Echo of the descriptive payload fields (null if not provided).
`processedAt`	string (ISO 8601) | null	When the transaction was processed.
`createdAt`	string (ISO 8601)	When the record was created.
`replayed`	boolean	`false` for a newly created transaction; `true` when your request matched one already recorded (see Idempotency).

Idempotency (avoid double-posting)

The endpoint is safe to retry. A transaction is treated as a **duplicate** (and the original record is returned instead of creating a new one) when:

1. The pair **(your source, `externalTransactionId`)** has already been seen — this is automatic and is all you need, **or**
2. *(advanced, optional)* you send an `idempotencyKey` that was already used for this entity.

In both cases you receive a `201` with the **existing** transaction, marked `"replayed": true` , and **the transaction is not recorded twice**. This means: if you get a network timeout, you can safely resend the exact same request.

A repeat must carry the **same amount and currency** as the transaction already recorded. A repeat whose amount or currency differs is **not** a replay — it is refused with `409 TRANSACTION_MISMATCH` , nothing is recorded, and `error.details` shows what you `sent` (and, for the same `externalTransactionId` , what is `recorded`). Do not retry it: contact Rival Payments so the difference is reconciled.

The same applies to transactions Rival Payments recorded for you by hand (for example, before your integration went live): when you send one of them with **your own transaction reference** as `externalTransactionId` and the **same amount and currency**, you receive it back with `"replayed": true`

and it is not recorded twice. If the amount differs from what we recorded, you receive `409 TRANSACTION_MISMATCH` and we reconcile it with you.

> Reusing the *same* `externalTransactionId` for a *different* entity is rejected with `409 DUPLICATE_TRANSACTION`. Keep your `externalTransactionId` globally unique per transaction.

---

3. Error handling

All errors use the same envelope as success responses, with the failure under `error`:

```
{
  "success": false,
  "statusCode": 400,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable description",
    "details": {}
  }
}
```

`details` is optional and, for validation errors, lists the specific fields that failed. You can detect failure by `success === false` (or the presence of `error`); `statusCode` echoes the HTTP status.

Status codes

HTTP	`code`	When
`201`	—	Success — transaction created (or replayed, see Idempotency).
`400`	`VALIDATION_ERROR`	Body failed validation (missing/invalid field, bad `amount`, unsupported `currency`), or the `companySlug` is unknown or not available.
`401`	`UNAUTHORIZED`	Missing, malformed, invalid, or revoked API key.
`403`	`FORBIDDEN`	Your key was used on a source path other than `whish`, or the request was made without a source API key.
`409`	`DUPLICATE_TRANSACTION`	The `externalTransactionId` was already used for a **different** entity. Do not retry — use a new reference.
`409`	`TRANSACTION_MISMATCH`	The `externalTransactionId` (or `idempotencyKey`) is already recorded with a **different amount or currency**. Nothing was recorded. Do not retry — contact Rival Payments.
`409`	`WRITE_CONFLICT`	Transient write conflict under concurrent load — safe to retry.
`500`	`INTERNAL_ERROR`	Unexpected server error. Retry with backoff; contact Rival Payments if persistent.

Recommended retry policy

- **Network errors / timeouts / `409 WRITE_CONFLICT` / `5xx`:** retry with exponential backoff. Because of idempotency, resending the identical request is safe.
- **`400` / `401` / `403` / `409 DUPLICATE_TRANSACTION` / `409 TRANSACTION_MISMATCH`:** do **not** blindly retry — the request itself is wrong; fix the payload/credentials/configuration first (for a mismatch, contact Rival Payments).

---

4. Example (cURL)

```
curl -sS -X POST \
  https://portal.rivalpayments.com/v1/ingest/whish/transactions \
  -H "Authorization: Bearer tsk_XXXXXXXX_REPLACE_WITH_YOUR_ISSUED_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "companySlug": "acme-retail",
    "externalTransactionId": "WISH-2026-000123456",
    "amount": "150.00",
    "currency": "USD",
    "payload": {
      "customer": "John Doe",
      "tradingAccount": "0096170123456",
      "phone": "0096170123456",
      "note": "Top-up via Whish"
    }
  }'
```

---

5. Integration summary

Everything you need is provided by Rival Payments:

1. **API key:** `tsk_XXXXXXXX_REPLACE_WITH_YOUR_ISSUED_KEY`
2. **Transaction URL:** `https://portal.rivalpayments.com/v1/ingest/whish/transactions`
3. **Entity slugs:** the `companySlug` values for the operating entities you may submit transactions for — we send you the list and keep you updated when it changes.

Document version 1.0